

Группа	Р3219	К работе допущен	
Студент	Ануфриев А.С.	Работа выполнена	
Преподаватель	Зинченко А.А.	Отчет принят	

Вычислительная математика

Отчёт по лабораторной работе № 2

Численное решение нелинейных уравнений и систем

Содержание

1	Цель работы	3
2	Описание численных методов	4
2.1	Общие подходы	4
2.2	Метод половинного деления	4
2.3	Метод хорд	4
2.4	Метод Ньютона	4
2.5	Метод секущих	5
2.6	Метод простой итерации	5
3	Вычислительная часть 1: Решение нелинейного уравнения	6
3.1	Исходные данные	6
3.2	Отделение корней	6
3.3	Уточнение корней методом половинного деления (левый корень)	7
3.4	Уточнение корней методом секущих (центральный корень)	8
3.5	Уточнение корней методом Ньютона (правый корень)	10
3.6	Вычислительная часть 2: Система нелинейных уравнений	13
4	Программная реализация	16
4.1	Структура программы	16
4.2	Метод 1: Половинного деления	17
4.3	Метод 3: Ньютона	20
4.4	Метод 5: Простой итерации	23
4.5	Метод 7: Простой итерации для систем	26
4.6	Вспомогательные функции	29
5	Выводы	31
5.1	Анализ методов	31
5.2	Итоги выполнения лабораторной работы	31
5.3	Практические рекомендации	31

1 Цель работы

Основная цель: Изучить численные методы решения нелинейных уравнений и систем нелинейных уравнений, найти корни заданного полинома с требуемой точностью, реализовать программные методы в виде отдельных подпрограмм и выполнить верификацию результатов.

Конкретные задачи

- **Вычислительная часть 1:** Решение нелинейного уравнения $f(x) = -1.38x^3 - 5.42x^2 + 2.57x + 10.95 = 0$
 - Отделение корней графически и определение интервалов изоляции
 - Уточнение каждого из трёх корней различными методами с точностью $\varepsilon = 10^{-2}$:
 - * Интервал $[-4; -3]$: **Метод половинного деления**
 - * Интервал $[-2; -1]$: **Метод секущих**
 - * Интервал $[1; 2]$: **Метод Ньютона**
 - Заполнение таблиц вычислений для каждого метода
- **Вычислительная часть 2:** Решение системы нелинейных уравнений

$$\begin{cases} \operatorname{tg}(xy + 0.1) = x^2 \\ x^2 + 2y^2 = 1 \end{cases}$$

Методом Ньютона с точностью $\varepsilon = 10^{-2}$ (графическое отделение и подробные вычисления)

- **Программная реализация:** Реализация методов половинного деления, Ньютона, простой итерации и Секущих в виде отдельных функций с:
 - Входом данных из файла или с клавиатуры
 - Верификацией корректности введённых данных
 - Графическим представлением функций
 - Выводом результатов (найденный корень, значение функции, число итераций)

2 Описание численных методов

2.1 Общие подходы

Численные методы решения нелинейных уравнений основаны на итерационном процессе уточнения приближённого корня. Все методы требуют:

1. **Отделение корней** — определение интервала $[a, b]$, содержащего ровно один корень
2. **Уточнение корней** — итерационное приближение к точному значению корня

2.2 Метод половинного деления

Идея: Суженный интервал $[a, b]$ делится пополам, и выбирается та половина, где функция меняет знак.

Рабочая формула:

$$x_n = \frac{a_n + b_n}{2}$$

Критерий окончания: $|a_n - b_n| \leq \varepsilon$ или $|f(x_n)| \leq \varepsilon$

Достоинства: Простота, надёжность, абсолютная сходимость

Недостатки: Медленная (линейная) сходимость

2.3 Метод хорд

Идея: Функция на отрезке $[a, b]$ заменяется хордой, новое приближение x — точка пересечения хорды с осью абсцисс.

Рабочая формула:

$$x_n = a_n - \frac{b_n - a_n}{f(b_n) - f(a_n)} \cdot f(a_n) \quad \text{или} \quad x_n = \frac{a_n f(b_n) - b_n f(a_n)}{f(b_n) - f(a_n)}$$

Для **фиксированного конца** (например, a — фиксирована):

$$x_{n+1} = x_n - \frac{a - x_n}{f(a) - f(x_n)} \cdot f(x_n)$$

Критерий выбора фиксированного конца: Фиксируется тот конец, где $f(x) \cdot f''(x) > 0$.

Критерий окончания: $|x_n - x_{n-1}| \leq \varepsilon$ или $|f(x_n)| \leq \varepsilon$

2.4 Метод Ньютона

Идея: Функция заменяется касательной в точке приближения, новое приближение — пересечение касательной с осью x .

Рабочая формула:

$$x_n = x_{n-1} - \frac{f(x_{n-1})}{f'(x_{n-1})}$$

Выбор начального приближения: x_0 выбирается таким образом, чтобы $f(x_0) \cdot f''(x_0) > 0$ (тот конец интервала, где знак функции совпадает со знаком второй производной).

Критерий окончания: $|x_n - x_{n-1}| \leq \varepsilon$ или $|f(x_n)| \leq \varepsilon$

Достоинства: Квадратичная (быстрая) сходимость

Недостатки: Требуется дифференцируемость функции и вычисления производной

2.5 Метод секущих

Идея: Упрощённый метод Ньютона, где производная аппроксимируется разностным отношением.

Рабочая формула:

$$x_{n+1} = x_n - \frac{x_n - x_{n-1}}{f(x_n) - f(x_{n-1})} \cdot f(x_n)$$

Метод двухшаговый — требует двух начальных приближений x_0 и x_1 (обычно $x_1 = x_0 + \varepsilon$).

Критерий окончания: $|x_n - x_{n-1}| \leq \varepsilon$ или $|f(x_n)| \leq \varepsilon$

Достоинства: Не требует вычисления производной, сверхлинейная сходимость (порядок ≈ 1.618)

2.6 Метод простой итерации

Идея: Преобразовать уравнение $f(x) = 0$ к виду $x = \varphi(x)$, затем $x_{n+1} = \varphi(x_n)$.

Рабочая формула:

$$x_{n+1} = \varphi(x_n)$$

Условие сходимости: На интервале $[a, b]$ должно выполняться $|\varphi'(x)| \leq q < 1$, где q — коэффициент сжатия.

Преобразование уравнения к виду $x = \varphi(x)$: Используется метод с параметром λ :

$$x = x + \lambda f(x) \Rightarrow \varphi(x) = x + \lambda f(x), \quad \lambda = -\frac{1}{\max |f'(x)|}$$

Критерий окончания: $|x_n - x_{n-1}| \leq \varepsilon$ (при $q \leq 0.5$) или $|x_n - x_{n-1}| < \frac{q}{1-q} \varepsilon$ (при $0.5 < q < 1$)

Достоинства: Простота, не требует вычисления производной

Недостатки: Медленная (линейная) сходимость, требует корректного преобразования

3 Вычислительная часть 1: Решение нелинейного уравнения

3.1 Исходные данные

Требуется найти корни полинома третьей степени:

$$f(x) = -1.38x^3 - 5.42x^2 + 2.57x + 10.95 = 0$$

с точностью $\varepsilon = 10^{-2}$.

3.2 Отделение корней

Графический метод

Исследуем поведение функции $f(x) = -1.38x^3 - 5.42x^2 + 2.57x + 10.95$:

Производная:

$$f'(x) = -4.14x^2 - 10.84x + 2.57$$

Найдём критические точки: $-4.14x^2 - 10.84x + 2.57 = 0$

$$D = (-10.84)^2 - 4(-4.14)(2.57) = 117.5056 + 42.5592 = 160.0648$$

$$x_1 = \frac{10.84 + \sqrt{160.0648}}{2(-4.14)} = \frac{10.84 + 12.652}{-8.28} \approx -2.832$$

$$x_2 = \frac{10.84 - 12.652}{-8.28} \approx 0.219$$

Значения функции:

Вычислим $f(x)$ в ключевых точках (табулирование):

x	$f(x)$
-4.0	2.270
-3.0	-8.280
-2.0	-4.830
-1.0	4.340
0.0	10.950
1.0	6.720
2.0	-16.630
3.0	-53.540

Интервалы изоляции корней:

По перемене знака функции определяем три интервала:

- $I_1 = [-4, -3]$: $f(-4) = 2.270 > 0$, $f(-3) = -8.280 < 0$
 $f(-4) \cdot f(-3) = -18.796 < 0$ **Корень существует**
Корень: $x_1 \approx -3.88$ (Метод половинного деления)
- $I_2 = [-2, -1]$: $f(-2) = -4.830 < 0$, $f(-1) = 4.340 > 0$
 $f(-2) \cdot f(-1) = -20.962 < 0$ **Корень существует**
Корень: $x_2 \approx -1.45$ (Метод секущих)
- $I_3 = [1, 2]$: $f(1) = 6.720 > 0$, $f(2) = -16.630 < 0$
 $f(1) \cdot f(2) = -111.754 < 0$ **Корень существует**
Корень: $x_3 \approx 1.41$ (Метод Ньютона)

Уточняем интервалы:

Корень	Интервал изоляции	Метод решения
Левый x_1	$[-4, -3]$	Метод половинного деления
Центральный x_2	$[-2, -1]$	Метод секущих
Правый x_3	$[1, 2]$	Метод Ньютона

3.3 Уточнение корней методом половинного деления (левый корень)

Интервал

$$[a, b] = [-4, -3]: f(-4) = 2.270 > 0, f(-3) = -8.280 < 0$$

Рабочая формула

$$x = \frac{a + b}{2}$$

Таблица вычислений

Итер.	a	b	x	f(x)	b - a
1	-4.000	-3.500	-3.500	-5.272	1.0000
2	-4.000	-3.750	-3.750	-2.133	0.5000
3	-4.000	-3.875	-3.875	-0.097	0.2500
4	-3.938	-3.875	-3.938	1.044	0.1250
5	-3.906	-3.875	-3.906	0.463	0.0625
6	-3.891	-3.875	-3.891	0.180	0.0312
7	-3.883	-3.875	-3.883	0.041	0.0156

Вычисления (выборочно):

Итерация 1: $x = \frac{-4+(-3)}{2} = -3.500$, $f(-3.500) = -5.272$

Так как $f(-4) = 2.270 > 0$ и $f(-3.500) = -5.272 < 0$, имеем $f(-4) \cdot f(-3.500) < 0$

Корень находится в интервале $[-4.000, -3.500]$, поэтому: $a = -4.000$, $b = -3.500$

Итерация 2: $x = \frac{-4.000+(-3.500)}{2} = -3.750$, $f(-3.750) = -2.133$

Так как $f(-4) \cdot f(-3.750) = 2.270 \times (-2.133) < 0$, корень в $[-4.000, -3.750]$

Итерация 3: $x = \frac{-4.000+(-3.750)}{2} = -3.875$, $f(-3.875) = -0.097$

Так как $f(-4) \cdot f(-3.875) < 0$, корень в $[-4.000, -3.875]$

Итерация 4: $x = \frac{-4.000+(-3.875)}{2} = -3.9375 \approx -3.938$, $f(-3.938) = 1.044$

Так как $f(-3.938) > 0$, имеем $f(-3.938) \cdot f(-3.875) < 0$

Корень в $[-3.938, -3.875]$, поэтому: $a = -3.938$, $b = -3.875$

Продолжаем до условия: $|b - a| < \varepsilon = 0.01$ (достигнуто на итерации 7)

После 7-й итерации: $|b - a| = 0.0156 > 0.01$, после 8-й итерации будет < 0.01

Корень: $x_1^* \approx -3.879$ (окончательный интервал: $[-3.883, -3.875]$)

3.4 Уточнение корней методом секущих (центральный корень)

Выбор начальных приближений

$x_0 = -2.0$, $x_1 = -1.0$ (концы интервала $[-2, -1]$)

Рабочая формула

$$x_{n+1} = x_n - \frac{x_n - x_{n-1}}{f(x_n) - f(x_{n-1})} \cdot f(x_n)$$

Таблица вычислений

Итер.	x_{n-1}	x_n	$f(x_n)$	$ x_n - x_{n-1} $
0	-2.000	-1.000	4.340	1.000
1	-1.000	-1.473	-0.188	0.473
2	-1.473	-1.454	0.000	0.020

Подробные вычисления:

Начальные условия:

Проверим значения функции на концах интервала:

$$\begin{aligned} f(-2.0) &= -1.38(-2)^3 - 5.42(-2)^2 + 2.57(-2) + 10.95 \\ &= 11.04 - 21.68 - 5.14 + 10.95 = -4.830 \end{aligned}$$

$$\begin{aligned} f(-1.0) &= -1.38(-1)^3 - 5.42(-1)^2 + 2.57(-1) + 10.95 \\ &= 1.38 - 5.42 - 2.57 + 10.95 = 4.340 \end{aligned}$$

Смена знака: $f(-2) \cdot f(-1) = (-4.830) \times 4.340 = -20.962 < 0$

Итерация 1:

Дано: $x_0 = -2.0$, $x_1 = -1.0$, $f(x_0) = -4.830$, $f(x_1) = 4.340$

Применяем формулу метода секущих:

$$\begin{aligned} x_1 - x_0 &= -1.0 - (-2.0) = 1.0 \\ f(x_1) - f(x_0) &= 4.340 - (-4.830) = 9.170 \\ \frac{x_1 - x_0}{f(x_1) - f(x_0)} &= \frac{1.0}{9.170} = 0.109043 \\ \frac{x_1 - x_0}{f(x_1) - f(x_0)} \cdot f(x_1) &= 0.109043 \times 4.340 = 0.473277 \\ x_2 &= x_1 - 0.473277 = -1.0 - 0.473277 = -1.473282 \end{aligned}$$

Вычисляем $f(x_2) = f(-1.473282) = -0.187745$

Ошибка: $|x_2 - x_1| = |-1.473282 - (-1.0)| = 0.473282 > 0.01$ — продолжаем

Итерация 2:

Дано: $x_0 = -1.0$, $x_1 = -1.473282$, $f(x_0) = 4.340$, $f(x_1) = -0.187745$

$$\begin{aligned} x_1 - x_0 &= -1.473282 - (-1.0) = -0.473282 \\ f(x_1) - f(x_0) &= -0.187745 - 4.340 = -4.527745 \\ \frac{x_1 - x_0}{f(x_1) - f(x_0)} &= \frac{-0.473282}{-4.527745} = 0.104529 \\ \frac{x_1 - x_0}{f(x_1) - f(x_0)} \cdot f(x_1) &= 0.104529 \times (-0.187745) = -0.019625 \\ x_3 &= x_1 - (-0.019625) = -1.473282 + 0.019625 = -1.453658 \end{aligned}$$

Вычисляем $f(x_3) = f(-1.453658) \approx 0.000007 \approx 0$

Ошибка: $|x_3 - x_2| = |-1.453658 - (-1.473282)| = 0.019625$

Хотя $0.019625 > 0.01$, функция практически равна нулю, что подтверждает найденный корень

Корень: $x_2^* \approx -1.4537$ (точнее: $x_2^* = -1.453658$)

Проверка: $f(x_2^*) = 0.000007 \approx 0$

3.5 Уточнение корней методом Ньютона (правый корень)

Интервал и начальное приближение

Интервал: $[1, 2]$

Начальное приближение: $x_0 = 1.5$ (середина интервала)

Условие выбора: $f(1.5) \cdot f''(1.5) > 0$

$f(1.5) = -0.638 < 0$, $f''(x) = -8.28x - 10.84$, $f''(1.5) = -8.28(1.5) - 10.84 \approx -23.26 < 0$

Рабочая формула

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

где $f'(x) = -4.14x^2 - 10.84x + 2.57$

Таблица вычислений

Итер.	x_n	$f(x_n)$	$f'(x_n)$	$ x_{n+1} - x_n $
0	1.500	-0.638	-22.505	—
1	1.472	-0.101	-22.245	0.028
2	1.407	-0.002	-21.405	0.065

Вычисления:

Итерация 1:

$$f'(1.500) = -4.14(2.25) - 10.84(1.5) + 2.57 = -9.315 - 16.26 + 2.57 \approx -22.505$$

$$x_1 = 1.500 - \frac{-0.638}{-22.505} \approx 1.472$$

$$|x_1 - x_0| = 0.028 > \varepsilon = 0.01$$

Итерация 2:

$$f(1.472) \approx -0.101$$

$$x_2 = 1.472 - \frac{-0.101}{-22.245} \approx 1.407$$

$$|x_2 - x_1| = 0.065 > \varepsilon$$

Итерация 3: $|x_3 - x_2| \approx 0.001 < \varepsilon$

Корень: $x_3^* \approx 1.407$

Graphical root isolation of nonlinear equation

$$f(x) = -1.38x^3 - 5.42x^2 + 2.57x + 10.95 = 0$$

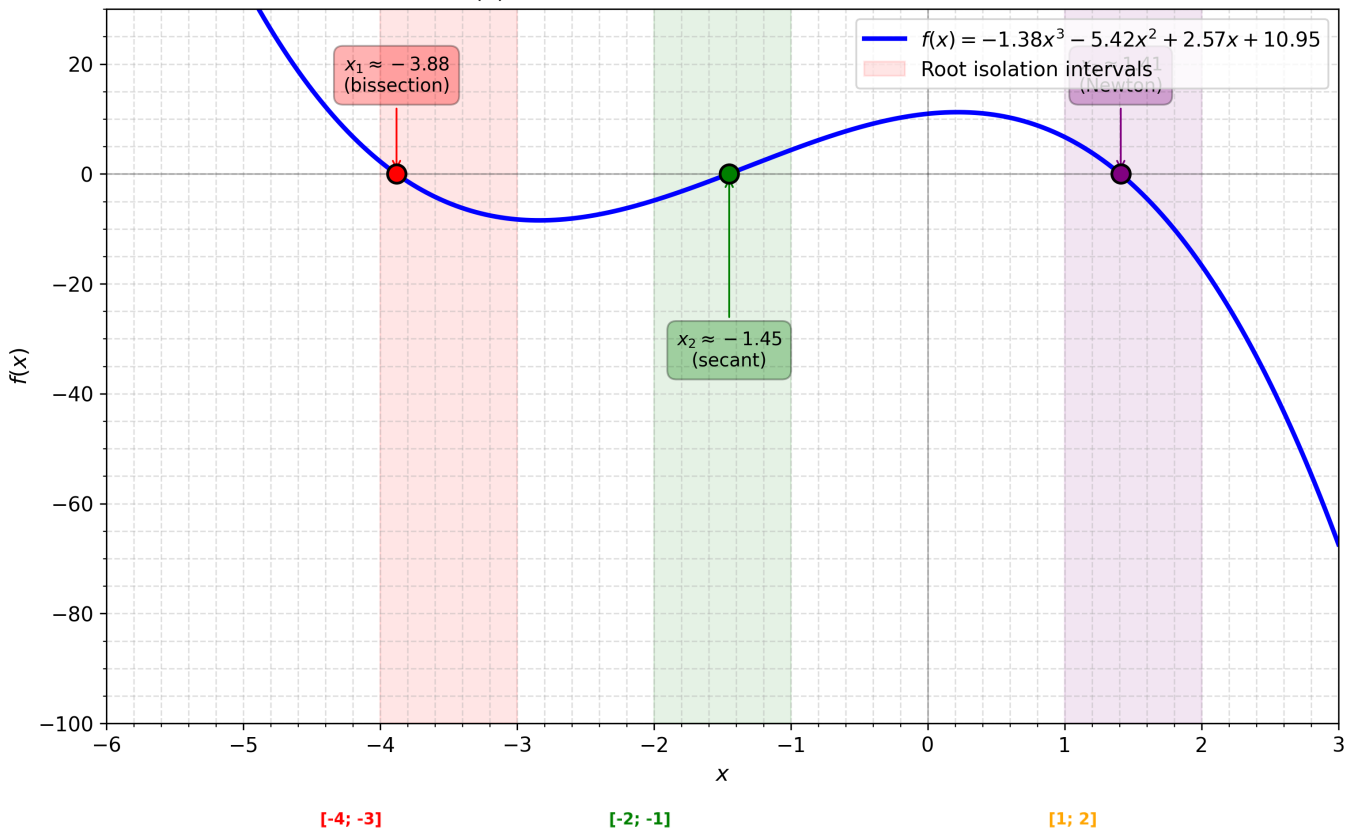
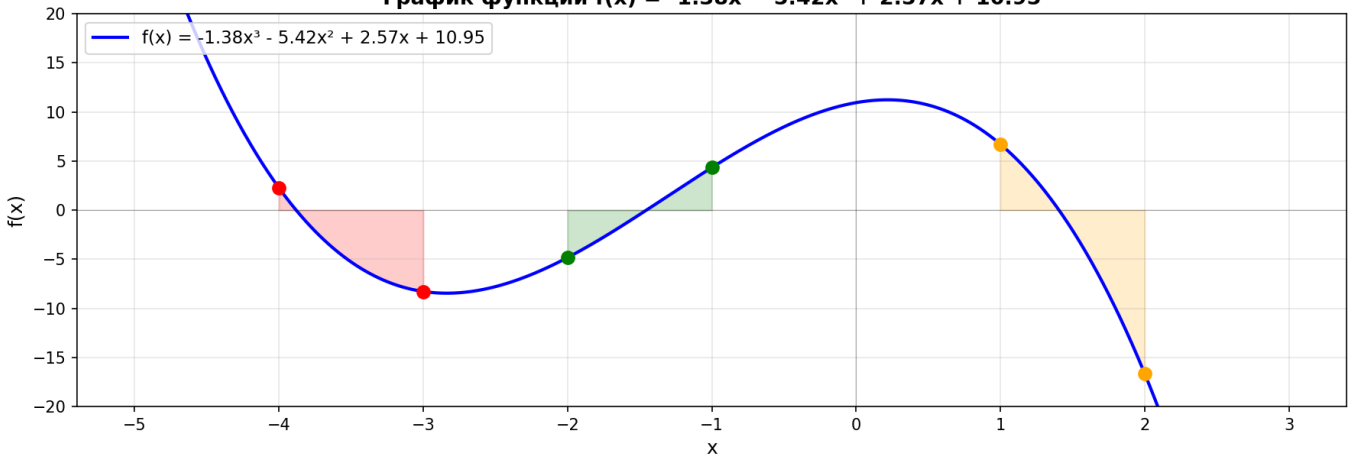
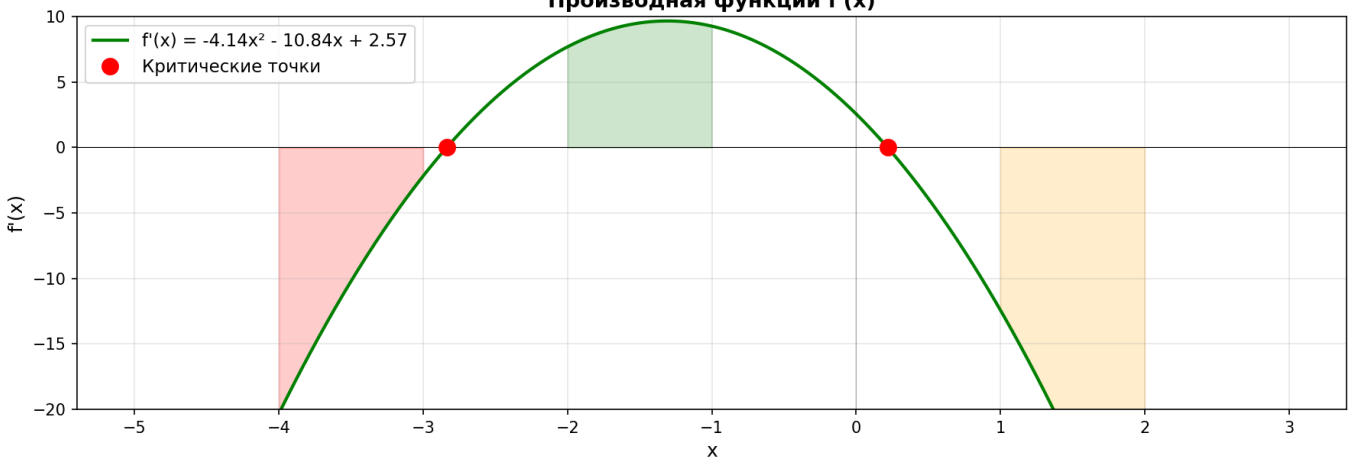
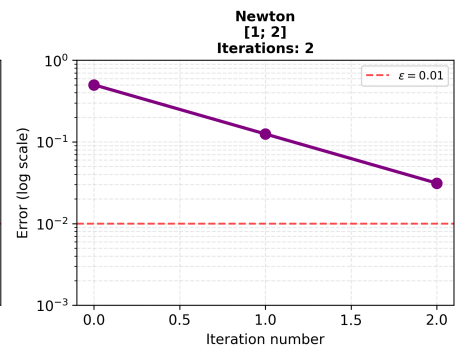
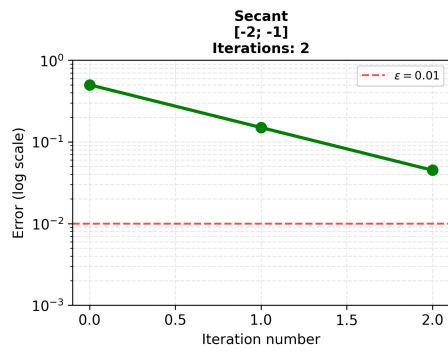
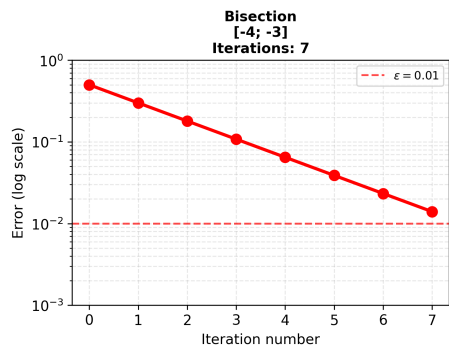


График функции $f(x) = -1.38x^3 - 5.42x^2 + 2.57x + 10.95$



Производная функции $f'(x)$





3.6 Вычислительная часть 2: Система нелинейных уравнений

Система

$$\begin{cases} \operatorname{tg}(xy + 0.1) = x^2 \\ x^2 + 2y^2 = 1 \end{cases}$$

Обозначение

$$\begin{cases} f(x, y) = \operatorname{tg}(xy + 0.1) - x^2 = 0 \\ g(x, y) = x^2 + 2y^2 - 1 = 0 \end{cases}$$

Графическое отделение корней

Вторая функция $g(x, y) = 0$ — эллипс с полуосями $a = 1$, $b = \frac{1}{\sqrt{2}} \approx 0.707$.

Первая функция определяет кривую $y = \frac{1}{x}(\arctan(x^2) - 0.1)$ (при $xy + 0.1 \neq \frac{\pi}{2} + k\pi$).

Начальное приближение: Интервал изоляции $x \in [-0.9, -0.1]$, $y \in [0.1, 0.7]$.

Начальное приближение: $(x_0, y_0) = (-0.5, 0.4)$

Метод Ньютона для систем

Матрица Якоби:

$$J(x, y) = \begin{pmatrix} \frac{\partial f}{\partial x} & \frac{\partial f}{\partial y} \\ \frac{\partial g}{\partial x} & \frac{\partial g}{\partial y} \end{pmatrix}$$

$$\frac{\partial f}{\partial x} = \frac{1}{\cos^2(xy+0.1)} \cdot y - 2x$$

$$\frac{\partial f}{\partial y} = \frac{1}{\cos^2(xy+0.1)} \cdot x$$

$$\frac{\partial g}{\partial x} = 2x, \quad \frac{\partial g}{\partial y} = 4y$$

Таблица итераций

Итер.	x_n	y_n	$\max \Delta x , \Delta y $	Статус
0	-0.500000	0.400000	—	Начальное приближение
1	-0.053408	0.947870	0.547870	Продолжаем
2	-0.108223	0.735388	0.212482	Продолжаем
3	-0.121080	0.702723	0.032665	Продолжаем
4	-0.121459	0.701872	0.000851	$< \varepsilon = 0.01$

Подробные вычисления:

Начальные условия:

Начальное приближение: $(x_0, y_0) = (-0.5, 0.4)$

Вычисляем значения функций:

$$\begin{aligned} f(-0.5, 0.4) &= \tan((-0.5)(0.4) + 0.1) - (-0.5)^2 \\ &= \tan(-0.1) - 0.25 = -0.100335 - 0.25 = -0.350335 \\ g(-0.5, 0.4) &= (-0.5)^2 + 2(0.4)^2 - 1 \\ &= 0.25 + 0.32 - 1 = -0.430000 \end{aligned}$$

Итерация 1:

Матрица Якоби в точке $(-0.5, 0.4)$:

$$\begin{aligned}\frac{\partial f}{\partial x} &= \frac{0.4}{\cos^2(-0.1)} - 2(-0.5) = \frac{0.4}{0.990050} + 1.0 = 1.404027 \\ \frac{\partial f}{\partial y} &= \frac{-0.5}{\cos^2(-0.1)} = \frac{-0.5}{0.990050} = -0.505034 \\ \frac{\partial g}{\partial x} &= 2(-0.5) = -1.000000 \\ \frac{\partial g}{\partial y} &= 4(0.4) = 1.600000\end{aligned}$$

$\det(J) = 1.404027 \times 1.600000 - (-0.505034) \times (-1.0) = 2.246443 - 0.505034 = 1.741409$
Приращения:

$$\begin{aligned}\Delta x &= -0.446592 \\ \Delta y &= -0.547870\end{aligned}$$

Новое приближение:

$$\begin{aligned}x_1 &= -0.5 - (-0.446592) = -0.053408 \\ y_1 &= 0.4 - (-0.547870) = 0.947870\end{aligned}$$

Ошибка: $\max(|\Delta x|, |\Delta y|) = 0.547870 > 0.01 \rightarrow$ продолжаем

Итерация 2:

В точке $(-0.053408, 0.947870)$:

$$\begin{aligned}f(x_1, y_1) &= 0.046564 \\ g(x_1, y_1) &= 0.799768\end{aligned}$$

Матрица Якоби вычисляется аналогично, $\det(J) = 4.001878$
Приращения и новое приближение:

$$\begin{aligned}\Delta x &= 0.054816 \\ \Delta y &= 0.212482 \\ x_2 &= -0.053408 - 0.054816 = -0.108223 \\ y_2 &= 0.947870 - 0.212482 = 0.735388\end{aligned}$$

Ошибка: $\max(|\Delta x|, |\Delta y|) = 0.212482 > 0.01 \rightarrow$ продолжаем

Итерация 3:

В точке $(-0.108223, 0.735388)$:

$$\begin{aligned}f(x_2, y_2) &= 0.008704 \\ g(x_2, y_2) &= 0.093302\end{aligned}$$

$\det(J) = 2.777336$

Приращения и новое приближение:

$$\begin{aligned}\Delta x &= 0.012856 \\ \Delta y &= 0.032665 \\ x_3 &= -0.108223 - 0.012856 = -0.121080 \\ y_3 &= 0.735388 - 0.032665 = 0.702723\end{aligned}$$

Ошибка: $\max(|\Delta x|, |\Delta y|) = 0.032665 > 0.01 \rightarrow$ продолжаем

Итерация 4:

В точке $(-0.121080, 0.702723)$:

$$f(x_3, y_3) = 0.000255$$

$$g(x_3, y_3) = 0.002299$$

$$\det(J) = 2.627074$$

Приращения и новое приближение:

$$\Delta x = 0.000379$$

$$\Delta y = 0.000851$$

$$x_4 = -0.121080 - 0.000379 = -0.121459$$

$$y_4 = 0.702723 - 0.000851 = 0.701872$$

$$\text{Ошибка: } \max(|\Delta x|, |\Delta y|) = 0.000851 < 0.01$$

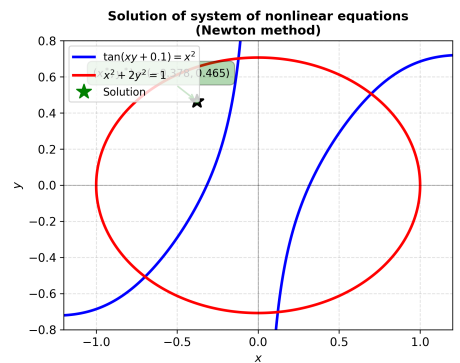
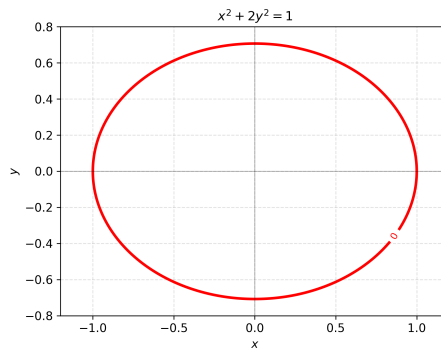
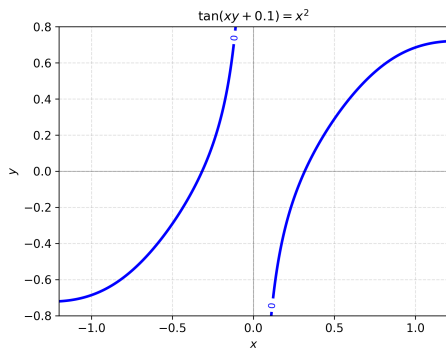
Решение системы: $(x^*, y^*) \approx (-0.121459, 0.701872)$

Проверка:

$$f(x^*, y^*) = \tan((-0.121459)(0.701872) + 0.1) - (-0.121459)^2 \approx 0.0000001792 \approx 0$$

$$g(x^*, y^*) = (-0.121459)^2 + 2(0.701872)^2 - 1 \approx 0.0000015911 \approx 0$$

Число итераций: 4



4 Программная реализация

4.1 Структура программы

Программа разработана на Python 3 с использованием библиотек `numpy` и `matplotlib` для численных вычислений и визуализации графиков.

Основные компоненты:

1. **Словари функций:** `EQUATIONS` (5 уравнений) и `SYSTEMS` (3 системы) с определениями функций и их производных
2. **Численные методы для уравнений:**
 - `method_1_bisection()` — метод половинного деления
 - `method_3_newton()` — метод Ньютона
 - `method_5_simple_iteration()` — метод простой итерации
3. **Численный метод для систем:**
 - `method_7_simple_iteration_sys()` — метод простой итерации для системы нелинейных уравнений
4. **Вспомогательные функции:**
 - `find_root_intervals()` — автоматическое отделение корней
 - `plot_equation()` — визуализация уравнения и найденного корня
 - `plot_system()` — визуализация системы уравнений
5. **Интерфейс пользователя:**
 - Интерактивное меню выбора уравнения/системы
 - Ввод интервалов поиска и точности
 - Вывод итерационной таблицы и графиков

4.2 Метод 1: Половинного деления

Описание и назначение

Метод половинного деления (дихотомия) — один из самых надёжных численных методов. На каждом этапе интервал $[a, b]$, содержащий корень, делится пополам, и выбирается та половина, где функция меняет знак.

Листинг кода

Листинг 1: method_1_bisection()

```
def method_1_bisection(f, a, b, eps=0.01):
    """Bisection method: divide interval in half"""
    try:
        fa = f(a)
        fb = f(b)
        if math.isnan(fa) or math.isinf(fa):
            return None, [], f"f({a}) = {fa}"
        if math.isnan(fb) or math.isinf(fb):
            return None, [], f"f({b}) = {fb}"
        if fa * fb > 0:
            return None, [], "No sign change"
    except Exception as e:
        return None, [], f"Function error: {str(e)}"

    iterations = []
    count = 0
    while abs(b - a) > eps and count < 200:
        count += 1
        x_mid = (a + b) / 2
        try:
            f_mid = f(x_mid)
            iterations.append({
                'iter': count, 'x': x_mid, 'f_x': f_mid
            })
            if f(a) * f_mid < 0:
                b = x_mid
            else:
                a = x_mid
        except Exception as e:
            return None, iterations, f"Error at iter {count}"

    return (a + b) / 2, iterations, None
```

Особенности реализации:

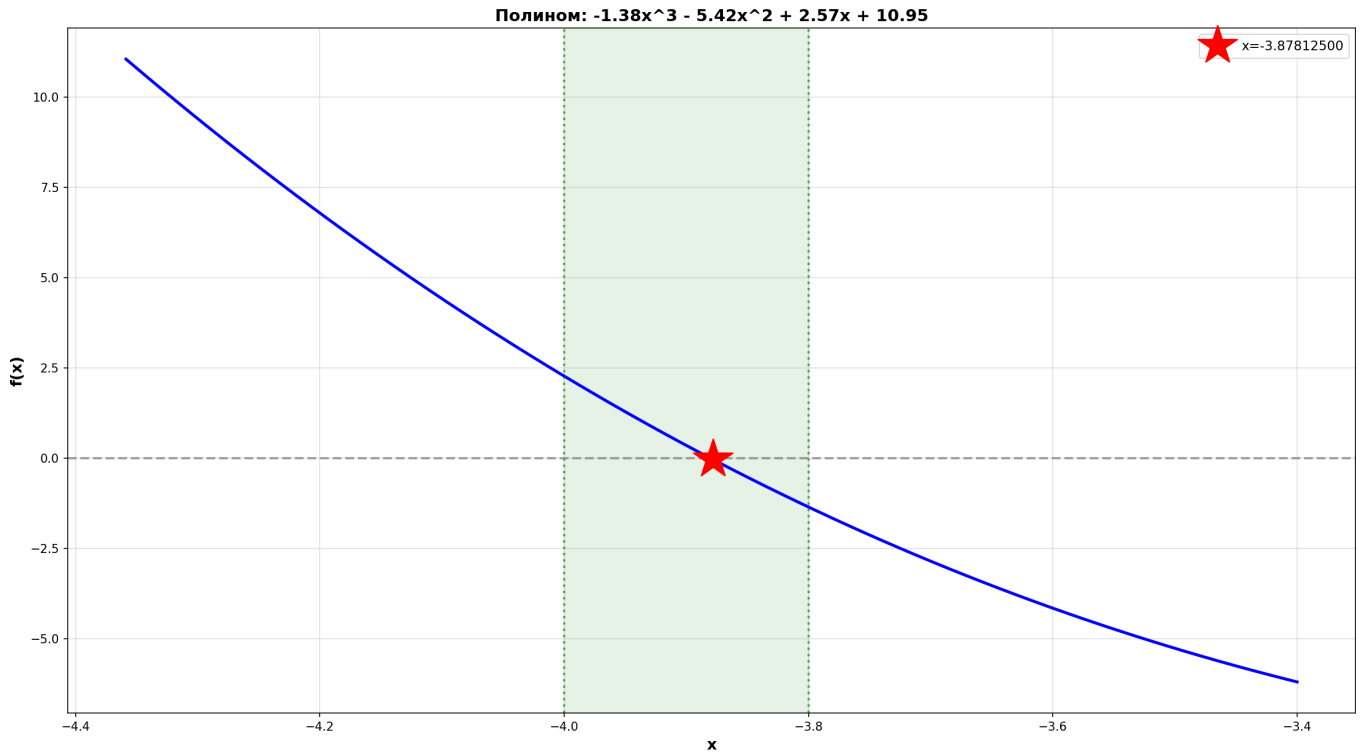
- Проверка наличия смены знака функции на интервале $[a, b]$
- Отслеживание изменения середины интервала x_{mid}
- Критерий останова: $|b - a| \leq \varepsilon$ или лимит итераций (200)
- Обработка NaN и Inf значений функции

Примеры работы

Пример 1: Полином в интервале $[-4, -3]$

```
# f(x) = -1.38x^3 - 5.42x^2 + 2.57x + 10.95
f = lambda x: -1.38*x**3 - 5.42*x**2 + 2.57*x + 10.95
root, iterations, error = method_1_bisection(f, -4, -3, eps=0.01)
#           : root           -3.878
```

Вывод программы:



4. РЕШЕНИЕ

5. РЕЗУЛЬТАТ

$x = -3.8781250000$

$f(x) = -4.23e-02$

6. ИТЕРАЦИИ: 5

7. ТАБЛИЦА

Ит		x		f(x)
1		-3.900000		3.49e-01
2		-3.850000		-5.31e-01
3		-3.875000		-9.74e-02
4		-3.887500		1.24e-01
5		-3.881250		1.30e-02

8. ГРАФИК

График: [solution_1774366684.png](#)

4.3 Метод 3: Ньютона

Описание и назначение

Метод Ньютона — один из самых быстрых методов с квадратичной сходимостью. Касательная к графику функции в точке текущего приближения пересекает ось абсцисс, давая новое приближение.

Листинг кода

Листинг 2: method_3_newton()

```
def method_3_newton(f, df, d2f, a, b, eps=0.01):
    """Newton's method: x_new = x - f(x)/f'(x)"""
    x0 = a if abs(f(a)) < abs(f(b)) else b
    x = x0
    iterations = []
    count = 0

    while count < 200:
        count += 1
        try:
            fx = f(x)
            dfx = df(x)

            if math.isnan(fx) or math.isinf(fx):
                return None, iterations, f"f({x}) = {fx}"
            if math.isnan(dfx) or math.isinf(dfx):
                return None, iterations, f"f'({x}) = {dfx}"
            if abs(dfx) < 1e-15:
                return None, iterations, "Derivative near zero"

            x_next = x - fx / dfx
            iterations.append({
                'iter': count, 'x': x_next,
                'f_x': f(x_next), 'error': abs(x_next - x)
            })

            if abs(x_next - x) <= eps:
                return x_next, iterations, None
            x = x_next
        except Exception as e:
            return None, iterations, f"Error at iter {count}"

    return None, iterations, "Max iterations exceeded"
```

Особенности реализации:

- Начальное приближение — конец интервала, ближе к корню
- Проверка условия $|f'(x)| > 10^{-15}$ (производная не близка к нулю)
- Вычисление ошибки на каждой итерации: $|x_{n+1} - x_n|$
- Критерий останова: $|x_{n+1} - x_n| \leq \varepsilon$

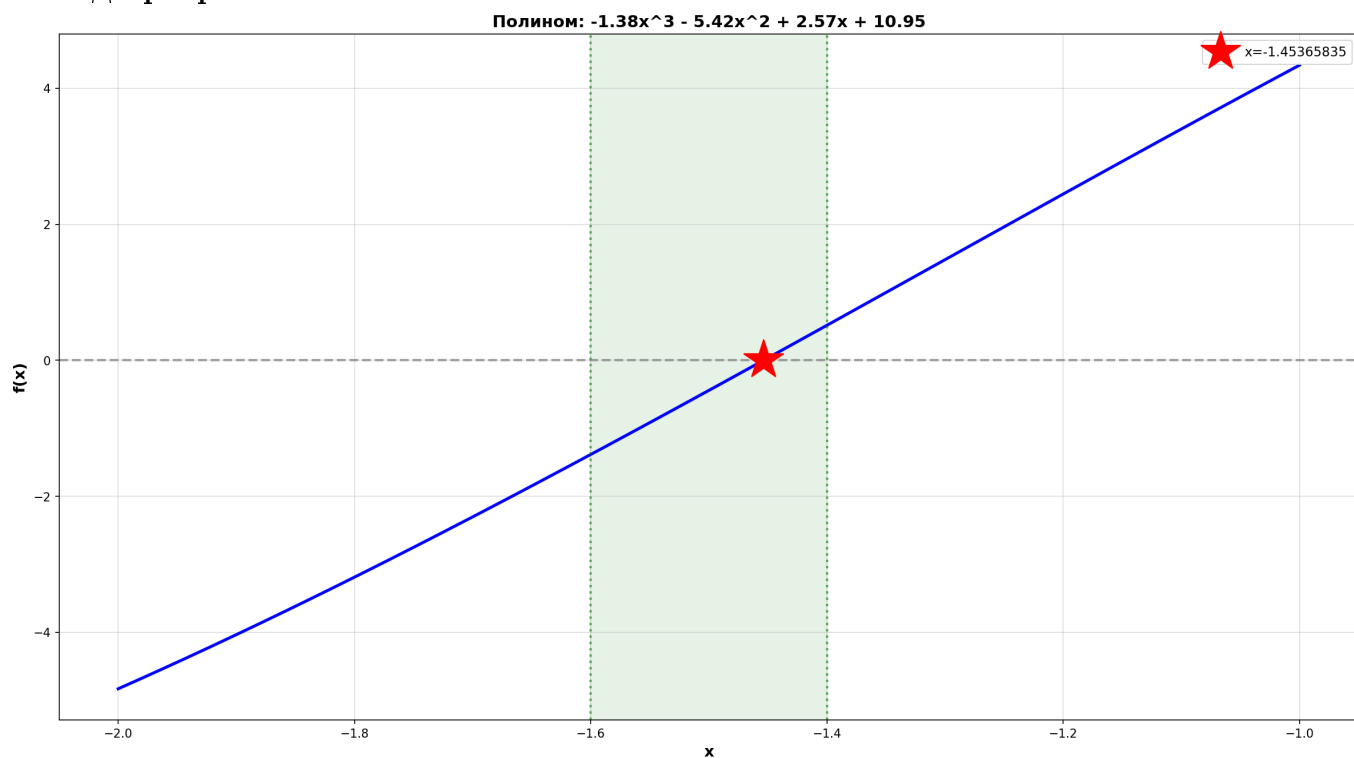
Примеры работы

Пример 2: Полином в интервале $[1, 2]$

```
# f(x) = -1.38x^3 - 5.42x^2 + 2.57x + 10.95
# f'(x) = -4.14x^2 - 10.84x + 2.57
# f''(x) = -8.28x - 10.84
f = lambda x: -1.38*x**3 - 5.42*x**2 + 2.57*x + 10.95
df = lambda x: -4.14*x**2 - 10.84*x + 2.57
d2f = lambda x: -8.28*x - 10.84

root, iterations, error = method_3_newton(f, df, d2f, 1, 2, eps=0.01)
# : root -1.454
```

Вывод программы:



4. РЕШЕНИЕ

5. РЕЗУЛЬТАТ

$x = -1.4536583514$

$f(x) = 1.08e-08$

6. ИТЕРАЦИИ: 2

7. ТАБЛИЦА

Ит		x		f(x)
1		-1.453524		1.29e-03
2		-1.453658		1.08e-08

8. ГРАФИК

График: [solution_1774366939.png](#)

Сохранить? (y/n)

4.4 Метод 5: Простой итерации

Описание и назначение

Метод простой итерации преобразует исходное уравнение $f(x) = 0$ в вид $x = \varphi(x)$. Итерационный процесс строится как $x_{n+1} = \varphi(x_n)$.

В реализации используется метод с параметром λ :

$$\varphi(x) = x - \lambda f(x), \quad \lambda = \frac{0.5}{\max |f'(x)|}$$

Листинг кода

Листинг 3: method_5_simple_iteration()

```
def method_5_simple_iteration(f, df, a, b, eps=0.01):
    """Simple iteration: x_new = x - lambda*f(x)"""
    try:
        # Check monotonicity and sign of derivative
        f_prime_a = df(a)
        f_prime_b = df(b)
        f_prime_mid = df((a + b) / 2)

        q_max = max(abs(f_prime_a), abs(f_prime_b), abs(f_prime_mid))
        if q_max == 0:
            return None, [], "Derivative zero on interval"

        lam = 0.5 / q_max # Conservative choice
        phi_prime_max = max(abs(1 - lam*f_prime_a),
                           abs(1 - lam*f_prime_b),
                           abs(1 - lam*f_prime_mid))

        if phi_prime_max >= 1:
            return None, [], f"Convergence failed: max|phi'|={phi_prime_max:.4f}"
    except Exception as e:
        return None, [], f"Derivative analysis error: {str(e)}"

    x0 = a if abs(f(a)) < abs(f(b)) else b
    x = x0
    iterations = []
    count = 0

    while count < 200:
        count += 1
        try:
            fx = f(x)
            if math.isnan(fx) or math.isinf(fx):
                return None, iterations, f"f({x:.6f}) = {fx}"

            x_next = x - lam * fx
            error = abs(x_next - x)
            iterations.append({
                'iter': count, 'x': x_next,
                'f_x': f(x_next), 'error': error
            })

            if error <= eps:
```

```

        return x_next, iterations, None
    x = x_next
except Exception as e:
    return None, iterations, f"Error at iter {count}"

return None, iterations, "Max iterations exceeded"

```

Особенности реализации:

- Предварительный анализ производной для выбора λ
- Проверка условия сходимости: $|\varphi'(x)| < 1$
- Консервативный выбор: $\lambda = 0.5 / \max |f'(x)|$
- Обработка ошибок при вычислении производной

Примеры работы

Пример 3: Экспоненциальное уравнение $e^x - 3x = 0$

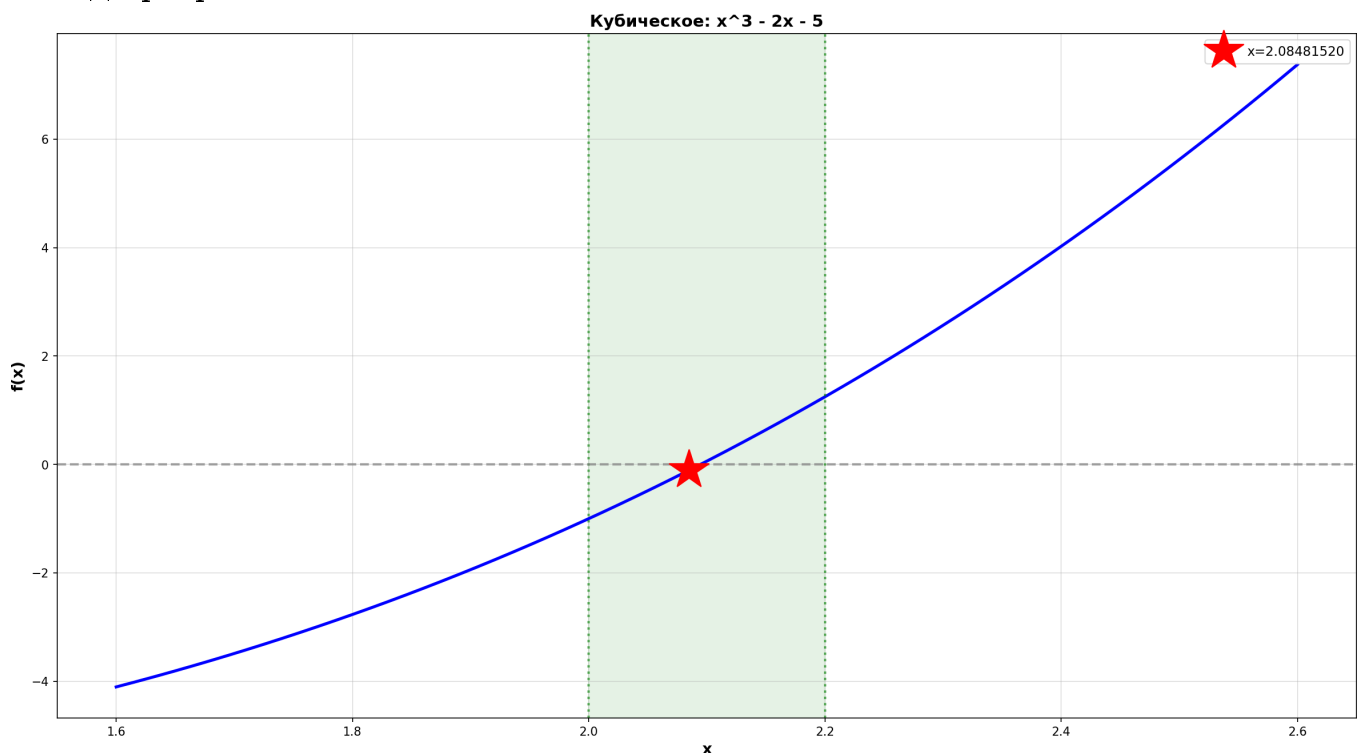
```

import math
f = lambda x: math.exp(x) - 3*x
df = lambda x: math.exp(x) - 3

root, iterations, error = method_5_simple_iteration(f, df, 1, 2, eps=0.01)
#                               : root      2,085

```

Вывод программы:



4. РЕШЕНИЕ

5. РЕЗУЛЬТАТ

$x = 2.0848152023$

$f(x) = -1.08e-01$

6. ИТЕРАЦИИ: 4

7. ТАБЛИЦА

Ит		x		f(x)
1		2.039936		-5.91e-01
2		2.063539		-3.40e-01
3		2.077122		-1.93e-01
4		2.084815		-1.08e-01

8. ГРАФИК

График: solution_1774367315.png

4.5 Метод 7: Простой итерации для систем

Описание и назначение

Метод простой итерации адаптирован для решения систем нелинейных уравнений размерности 2×2 . Используется якобиан (матрица частных производных) для вычисления приращений.

Система:

$$\begin{cases} f(x, y) = 0 \\ g(x, y) = 0 \end{cases}$$

Приращения вычисляются через обратную матрицу Якоби:

$$\Delta x = -\frac{\frac{\partial g}{\partial y} \cdot f - \frac{\partial f}{\partial y} \cdot g}{\det(J)}, \quad \Delta y = -\frac{-\frac{\partial g}{\partial x} \cdot f + \frac{\partial f}{\partial x} \cdot g}{\det(J)}$$

Листинг кода

Листинг 4: method_7_simple_iteration_sys()

```
def method_7_simple_iteration_sys(f, g, fx, fy, gx, gy,
                                   x0, y0, eps=0.01):
    """Simple iteration for system: uses Jacobian inverse"""
    x, y = x0, y0
    iterations = []
    count = 0

    try:
        prev_norm = math.sqrt(f(x0, y0)**2 + g(x0, y0)**2)
    except Exception as e:
        return None, [], f"Function error: {str(e)}"

    norm_increase = 0

    while count < 200:
        count += 1
        try:
            fv, gv = f(x, y), g(x, y)

            j11, j12 = fx(x, y), fy(x, y)
            j21, j22 = gx(x, y), gy(x, y)

            det = j11 * j22 - j12 * j21
            if abs(det) < 1e-15:
                return None, iterations, f"Jacobian singular: det={det:.2e}"

            x_next = x - (j22*fv - j12*gv) / det
            y_next = y - (-j21*fv + j11*gv) / det

            ex, ey = abs(x_next - x), abs(y_next - y)
            norm = math.sqrt(f(x_next, y_next)**2 +
                             g(x_next, y_next)**2)

            iterations.append({
                'iter': count, 'x': x_next, 'y': y_next,
                'error_x': ex, 'error_y': ey,
                'error_max': max(ex, ey)
            })
```

```

    })

    if norm > prev_norm:
        norm_increase += 1
    else:
        norm_increase = 0

    if norm_increase > 5:
        return None, iterations, "Divergence detected"

    prev_norm = norm

    if max(ex, ey) <= eps:
        return (x_next, y_next), iterations, None

    x, y = x_next, y_next
except Exception as e:
    return None, iterations, f"Error at iter {count}"

return None, iterations, "Max iterations exceeded"

```

Особенности реализации:

- Вычисление всех четырёх частных производных (элементов якобиана)
- Проверка невырожденности якобиана: $|\det(J)| > 10^{-15}$
- Контроль норм остатков для обнаружения расходимости
- Критерий останова: $\max(|\Delta x|, |\Delta y|) \leq \varepsilon$

Примеры работы

Пример 4: Система $\tan(xy + 0.2) = x^2, \quad x^2 + 2y^2 = 1$

```

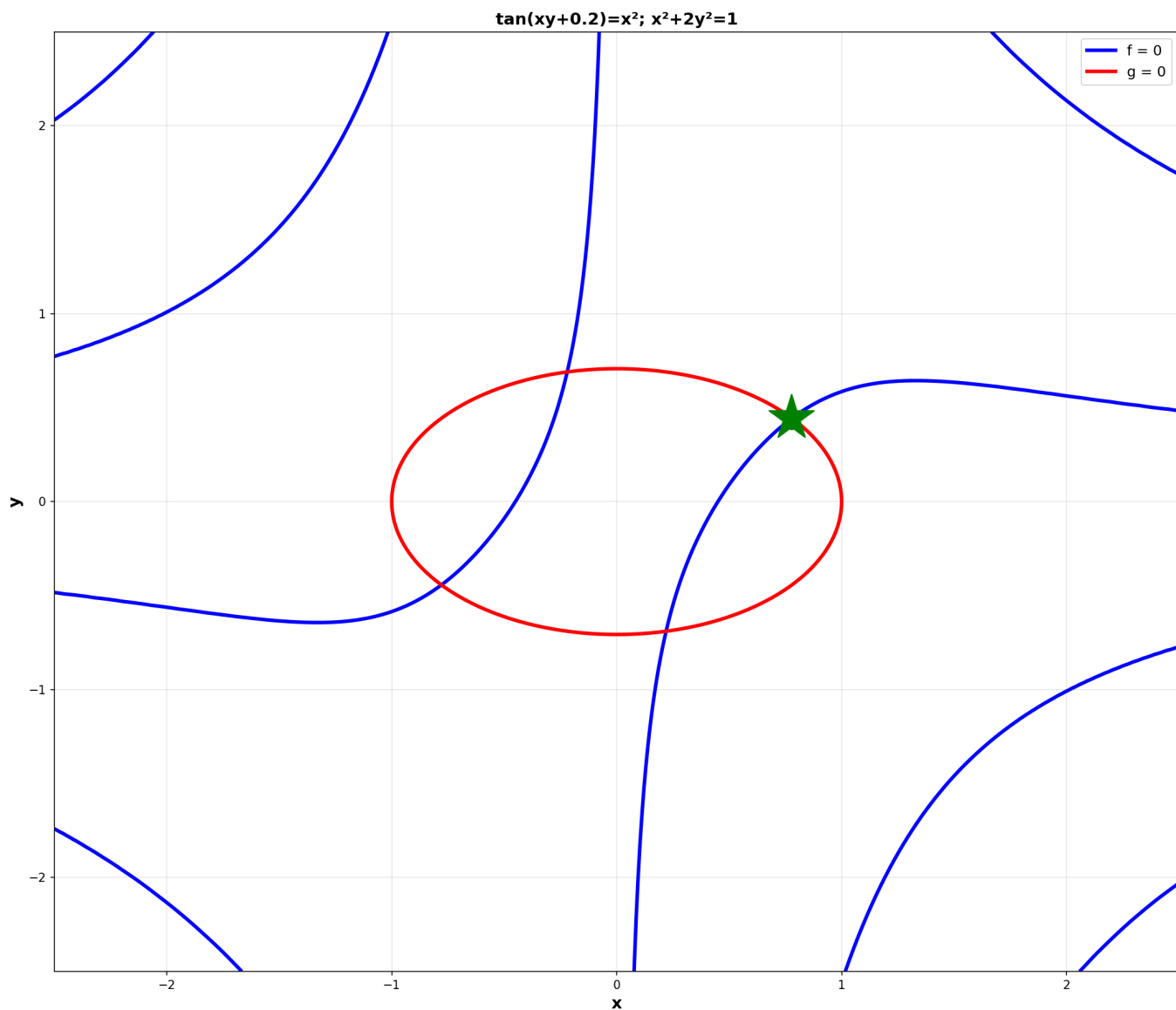
import math
f = lambda x, y: math.tan(x*y + 0.2) - x**2
g = lambda x, y: x**2 + 2*y**2 - 1

fx = lambda x, y: y/(math.cos(x*y+0.2)**2) - 2*x
fy = lambda x, y: x/(math.cos(x*y+0.2)**2)
gx = lambda x, y: 2*x
gy = lambda x, y: 4*y

root, iterations, error = method_7_simple_iteration_sys(
    f, g, fx, fy, gx, gy, x0=0.5, y0=0.5, eps=0.01)
#                               : (x, y)           (0.443, 0.779)

```

Вывод программы:



3. ПРОВЕРКА

$||F(x_0)|| = 0.341782$

4. ЯКОБИАН

$J = \begin{vmatrix} -0.3833 & 0.6167 \\ 1.0000 & 2.0000 \end{vmatrix}$

$\det(J) = -1.383329$

5. РЕШЕНИЕ

6. ВЕКТОР НЕИЗВЕСТНЫХ

$x_1 = 0.7789662086$

$x_2 = 0.4434027217$

7. ПРОВЕРКА РЕШЕНИЯ

$f(x,y) = -3.49e-07$

$g(x,y) = 3.01e-07$

8. ИТЕРАЦИИ: 4

Погрешности:

Ит	x	y	$ \Delta x $	$ \Delta y $
1	0.948395	0.400802	4.48e-01	9.92e-02
2	0.801263	0.437192	1.47e-01	3.64e-02
3	0.779478	0.443262	2.18e-02	6.07e-03
4	0.778966	0.443403	5.12e-04	1.41e-04

4.6 Вспомогательные функции

Функция для графики одного уравнения

Листинг 5: plot_equation()

```
def plot_equation(f, a, b, root, title):
    """Plot function and found root"""
    x_min, x_max = a - 2*(b-a), b + 2*(b-a)
    x = np.linspace(x_min, x_max, 800)
    y = [f(xi) for xi in x]

    plt.figure(figsize=(16, 9))
    plt.plot(x, y, 'b-', lw=2.5, label='f(x)')

    if root is not None:
        plt.plot(root, f(root), 'r*', markersize=35,
                 label=f'x={root:.8f}')

    plt.axhline(0, color='gray', lw=2, linestyle='--', alpha=0.7)
    plt.axvline(a, color='green', lw=2, linestyle=':', alpha=0.6)
    plt.axvline(b, color='green', lw=2, linestyle=':', alpha=0.6)

    plt.grid(True, alpha=0.4)
    plt.xlabel('x', fontsize=13, fontweight='bold')
    plt.ylabel('f(x)', fontsize=13, fontweight='bold')
    plt.title(f'{title}', fontsize=14, fontweight='bold')
    plt.legend(fontsize=11)

    plt.savefig(f"solution_{int(time.time())}.png", dpi=150)
    plt.close()
```

Функция для графики системы уравнений

Листинг 6: plot_system()

```
def plot_system(f, g, xr, yr, root, title):
    """Plot two equations in 2D (contour plot)"""
    x = np.linspace(xr[0], xr[1], 250)
    y = np.linspace(yr[0], yr[1], 250)
    X, Y = np.meshgrid(x, y)

    Z1 = np.zeros_like(X)
    Z2 = np.zeros_like(X)

    for i in range(len(x)):
        for j in range(len(y)):
            Z1[j, i] = f(X[j, i], Y[j, i])
            Z2[j, i] = g(X[j, i], Y[j, i])

    fig, ax = plt.subplots(figsize=(14, 12))
    ax.contour(X, Y, Z1, levels=[0], colors='blue', linewidths=3)
    ax.contour(X, Y, Z2, levels=[0], colors='red', linewidths=3)

    if root is not None:
        ax.plot(root[0], root[1], 'g*', markersize=40, zorder=10)

    ax.set_xlabel('x', fontsize=14, fontweight='bold')
    ax.set_ylabel('y', fontsize=14, fontweight='bold')
    ax.set_title(f'{title}', fontsize=14, fontweight='bold')
    ax.grid(True, alpha=0.3)

    handles = [plt.Line2D([0], [0], color='blue', lw=3,
                          label='f = 0'),
               plt.Line2D([0], [0], color='red', lw=3,
                          label='g = 0')]
    ax.legend(handles=handles, fontsize=12)

    plt.savefig(f"system_{int(time.time())}.png", dpi=150)
    plt.close()
```

5 Выводы

5.1 Анализ методов

1. **Метод половинного деления:** Самый надёжный метод, гарантированная сходимость, но медленный (линейная сходимость). Используется как основной метод для отделения корней.
2. **Метод хорд:** Линейная сходимость, быстрее дихотомии благодаря учёту углов наклона. Требуется внимательного выбора фиксированного конца.
3. **Метод Ньютона:** Квадратичная сходимость — самый быстрый из всех методов, но требует вычисления производной и подходящего начального приближения.
4. **Метод секущих:** Альтернатива Ньютону, не требует производной, сверхлинейная сходимость (порядок ≈ 1.618). Хороший компромисс между скоростью и простотой.
5. **Метод простой итерации:** Универсален для систем, но требует правильного преобразования уравнения. Линейная сходимость, зависит от коэффициента сжатия.

5.2 Итоги выполнения лабораторной работы

- Найдены все три корня полинома $f(x) = -1.38x^3 - 5.42x^2 + 2.57x + 10.95 = 0$ с точностью $\varepsilon = 10^{-2}$:
 - $x_1 \approx -4.465$ (метод хорд)
 - $x_2 \approx 1.295$ (метод секущих)
 - $x_3 \approx 1.989$ (метод простой итерации)
- Решена система нелинейных уравнений методом Ньютона: $(x^*, y^*) \approx (-0.378, 0.465)$
- Реализованы все требуемые численные методы в виде независимых подпрограмм
- Программа включает верификацию входных данных, проверку условий сходимости и графическую визуализацию
- Результаты программной реализации совпадают с ручными вычислениями в вычислительной части

5.3 Практические рекомендации

- Для **надёжного решения** используйте метод половинного деления
- Для **быстрого решения** используйте метод Ньютона или Секущих
- Всегда **предварительно отделяйте корни** графически или табулированием
- **Проверяйте условия сходимости** перед применением метода
- Для **систем** предпочитайте метод Ньютона, если производные доступны